



OPTIMIZACIÓN TRIDIMENSIONAL DE CARGA AEROPORTUARIA (3D BIN PACKING) CON RESTRICCIONES FÍSICAS Y DE BALANCE ESTÁTICO

Leonardo Duran Cuenca¹ Diego Villazón Arce²

¹Universidad Católica Boliviana “San Pablo” (Bolivia) email: leonardo.duran.c1@ucb.edu.bo

²Universidad Católica Boliviana “San Pablo” (Bolivia) email: diego.villazon@ucb.edu.bo

RESUMEN

El presente proyecto aborda el desafío logístico de cargar mercancía en aeronaves optimizando el valor económico transportado, garantizando simultáneamente la viabilidad geométrica y la seguridad del vuelo. Se formula y resuelve un modelo de Programación Lineal Entera Mixta (MILP) que combina una variante tridimensional del Problema de Empaquetado (*3D Bin Packing Problem*) y el Problema de la Mochila (*Knapsack Problem*). El modelo contempla restricciones críticas de la industria aeronáutica: dimensiones múltiples de compartimientos, límites de peso, reglas de no traslape espacial y el mantenimiento del Centro de Gravedad (CG) longitudinal dentro de rangos operativos seguros. La solución se implementó utilizando Pyomo con el solver CPLEX, integrada a una interfaz de renderizado 3D en el Frontend.

I. INTRODUCCIÓN

La logística aeroportuaria moderna exige maximizar las ganancias por flete minimizando los riesgos estructurales de las aeronaves. Asignar cajas a compartimientos de carga no es trivial debido a las estrictas restricciones geométricas y de peso.

El objetivo de este proyecto es desarrollar una herramienta de software (Backend + Frontend) que ingiera un inventario de cajas de distintos volúmenes y valores, y seleccione la mejor combinación para introducir en los compartimientos de un avión específico. Además de decidir qué cajas viajan, el sistema decide la

coordenada exacta (x, y, z) de cada caja garantizando que el peso distribuido no altere peligrosamente el momento estático (Centro de Gravedad) de la aeronave.

1.1. Marco Teórico y Estado del Arte

La Complejidad Computacional del 3D Bin Packing y Knapsack Problem

El problema abordado en este proyecto pertenece a la clase de problemas de optimización combinatoria y se clasifica como NP-hard (Non-Deterministic Polynomial-time hard). Al integrar el Problema de Empaquetado de Cajas Tridimensional (3D-BPP) con el Problema de la

Mochila (Knapsack Problem), la dificultad algorítmica se incrementa exponencialmente. El sistema no solo debe discriminar qué subconjunto de artículos maximiza el beneficio económico total bajo una restricción de capacidad lineal (lógica del Knapsack), sino que simultáneamente debe comprobar la viabilidad geométrica de la asignación en un espacio euclidiano tridimensional sin traslapes. Esto implica que el espacio de soluciones crece de forma factorial con respecto al número de cajas disponibles en el inventario (I), imposibilitando el uso de búsquedas exhaustivas o por fuerza bruta en tiempos de cómputo razonables.

Programación Lineal Entera Mixta (MILP) frente a Enfoques Heurísticos

En la logística aeroportuaria, es común encontrar la aplicación de algoritmos heurísticos tradicionales (como First-Fit Decreasing, algoritmos genéticos o búsqueda tabú) para la resolución del empaquetado tridimensional debido a su rapidez operativa. Sin embargo, las heurísticas no garantizan la optimización global y suelen fallar críticamente al intentar satisfacer restricciones físicas altamente restrictivas y acopladas. La elección de un modelo de Programación Lineal Entera Mixta (MILP) resuelto mediante un solver exacto como CPLEX garantiza encontrar la solución óptima global (o una solución con una brecha de optimalidad controlada). El uso del método disyuntivo y la técnica de la "M Grande" (Big-M) permite linealizar las condiciones lógicas de colisión

geométrica y asignación espacial. Aunque el enfoque MILP exige un mayor costo computacional, la precisión matemática asegura el aprovechamiento máximo de la capacidad instalada y elimina los márgenes de error que las heurísticas suelen arrastrar.

Fundamentos Aeronáuticos del Balance Estático y Centro de Gravedad (CG)

La distribución de carga en una aeronave no es meramente un problema de optimización volumétrica; es una condición crítica de aeronavegabilidad y seguridad de vuelo. El comportamiento aerodinámico y el control de un avión dependen estrictamente de la posición de su Centro de Gravedad (CG) a lo largo del eje longitudinal (brazo de palanca medido desde la nariz del avión).

Físicamente, el CG es el punto donde se concentran todas las fuerzas de peso de la aeronave. Para que un vuelo sea seguro y eficiente, el CG calculado tras la carga debe situarse estrictamente dentro de un rango operativo delimitado por un límite delantero (CG_{min}) y un límite trasero (CG_{max}) definido por el fabricante.

- **CG Adelantado ($< CG_{min}$):** Un avión excesivamente pesado en la nariz requiere que las superficies de control de la cola (elevators) generen una fuerza descendente masiva para mantener el vuelo nivelado. Esto incrementa la resistencia aerodinámica, eleva el

consumo de combustible y, en casos extremos, puede provocar que la aeronave no tenga suficiente autoridad de cabeceo para rotar durante el despegue.

- **CG Atrasado ($> CG_{max}$):** Un avión pesado en la cola reduce drásticamente la estabilidad longitudinal estática. La aeronave se vuelve altamente susceptible a perturbaciones, presenta una tendencia peligrosa al encabritamiento de forma descontrolada y puede entrar en pérdida (stall) de la cual el piloto no pueda recuperarse físicamente.

Por ende, la formulación matemática del modelo debe obligatoriamente cruzar las variables de decisión volumétricas locales con el cálculo del momento estático global (Suma de *peso* $\times$ *brazo de palanca*), garantizando de forma analítica que la aeronave despegue en perfecto equilibrio hidrostático.

II. FORMULACIÓN MATEMÁTICA (MODELO MILP)

El problema ha sido clasificado como Programación Lineal Entera Mixta (MILP). A continuación, se detalla la formulación técnica empleada:

2.1. Conjuntos y Parámetros

- **Conjuntos:**

- I : Conjunto de cajas disponibles en inventario ($i = 1, \dots, n$).
- J : Conjunto de compartimientos del avión seleccionado ($j = 1, \dots, m$).

```
model.I = pyo.Set(initialize=cajas_idx)
model.J = pyo.Set(initialize=comp_idx)
model.Pares = pyo.Set(initialize=[(i, k) for i in model.I for k in model.I if i < k])
```

- **Parámetros de las Cajas:**

- dx_i, dy_i, dz_i : Dimensiones físicas (ancho, largo, alto) de la caja i .
- w_i : Peso de la caja i .
- val_i : Valor económico (o prioridad) de la caja i .

```
model.dx = pyo.Param(model.I, initialize=dict_dx)
model.dy = pyo.Param(model.I, initialize=dict_dy)
model.dz = pyo.Param(model.I, initialize=dict_dz)
model.w = pyo.Param(model.I, initialize=dict_w)
model.val = pyo.Param(model.I, initialize=dict_val)
```

- **Parámetros de los Compartimientos:**

- $L_{X_j}, L_{Y_j}, L_{Z_j}$: Dimensiones máximas del compartimiento j .
- $Wmax_j$: Capacidad máxima de carga estructural (peso) del compartimiento j .
- $Ycomp_j$: Distancia desde la nariz del avión hasta el inicio del compartimiento j en el eje longitudinal.

```
model.x = pyo.Var(model.I, within=pyo.NonNegativeReals)
model.y = pyo.Var(model.I, within=pyo.NonNegativeReals)
model.z = pyo.Var(model.I, within=pyo.NonNegativeReals)
model.Y_abs = pyo.Var(model.I, within=pyo.NonNegativeReals)
```

- **Parámetros Globales:**

- CG_{min}, CG_{max} : Rango seguro del Centro de Gravedad.
- M : Constante "M Grande" (Big-M). Fijada estratégicamente en $M = 50$ para evitar errores de tolerancia fraccional del solver.

```
model.CG_min = pyo.Param(initialize=cg_min_global)
model.CG_max = pyo.Param(initialize=cg_max_global)
model.M = pyo.Param(initialize=50)
```

2.2. Variables de Decisión

- **Binarias:** $* V_{ij} \in \{0, 1\}$: 1 si la caja i es asignada al compartimiento j , 0 caso contrario.
 - Variables direccionales relativas para evitar traslapes: .

$left_{ik}, right_{ik}, front_{ik}, back_{ik}, below_{ik}, above_{ik}$
 $\in \{0, 1\}$.

```
model.V = pyo.Var(model.I, model.J, within=pyo.Binary)
model.left = pyo.Var(model.Pares, within=pyo.Binary)
model.right = pyo.Var(model.Pares, within=pyo.Binary)
model.front = pyo.Var(model.Pares, within=pyo.Binary)
model.back = pyo.Var(model.Pares, within=pyo.Binary)
model.below = pyo.Var(model.Pares, within=pyo.Binary)
model.above = pyo.Var(model.Pares, within=pyo.Binary)
```

- **Continuas:**

- $x_i, y_i, z_i \geq 0$: Coordenadas de la esquina inferior izquierda trasera de la caja i .
- $Y_{abs_i} \geq 0$: Coordenada longitudinal absoluta de la caja respecto a la nariz del avión.

```
model.L_X = pyo.Param(model.J, initialize=dict_LX)
model.L_Y = pyo.Param(model.J, initialize=dict_LY)
model.L_Z = pyo.Param(model.J, initialize=dict_LZ)
model.Wmax = pyo.Param(model.J, initialize=dict_Wmax)
model.Ycomp = pyo.Param(model.J, initialize=dict_Ycomp)
```

2.3. Función Objetivo y el Paradigma de “Gravedad Artificial”

El diseño de la función objetivo persigue dos metas simultáneas: una primaria (económica) y una secundaria (espacial/física). Aunque el modelo original es mono-objetivo, se implementó una técnica de ponderación matemática que simula una optimización lexicográfica, garantizando la viabilidad estructural del empaquetado tridimensional.

$$Max Z = \sum_{i \in I} \sum_{j \in J} (val_i \cdot V_{ij}) - \lambda \sum_{i \in I} (x_i + y_i + z_i)$$

```
# Objetivo: Maximizar valor y aplicar gravedad
def obj_rule(model):
    ganancia = sum(model.val[i] * model.V[i, j] for i in model.I for j in model.J)
    # Una pequeña penalización para que las cajas vayan al suelo y a la pared
    gravedad = 0.001 * sum(model.x[i] + model.y[i] + model.z[i] for i in model.I)
    return ganancia - gravedad
model.Obj = pyo.Objective(rule=obj_rule, sense=pyo.maximize)
```

2.3.1 Componente Primario: Maximización Económica

El primer término de la ecuación, $\sum(val_i \cdot V_{ij})$, corresponde a la variante clásica del Knapsack Problem. El solver buscará activar la variable binaria $V_{ij} = 1$ para aquellas cajas cuyo valor económico (o prioridad operativa) val_i maximice la sumatoria global, respetando siempre las restricciones de capacidad estructural y geométrica de cada compartimiento.

2.3.2 Componente Secundario: El Problema de la “Flotabilidad” Matemática

En la optimización combinatoria pura, un solver algebraico carece de un motor de físicas. Si la función objetivo se limitara únicamente a maximizar el valor económico, cualquier coordenada (x, y, z) que cumpla con los límites de la "M Grande" (Big-M) y no se traslape con otras cajas sería considerada una solución óptima válida. Esto genera un problema de múltiples óptimos degenerados, donde el solver puede arrojar resultados en los que las cajas "flotan" virtualmente en el centro del compartimiento o pegadas al techo ($z > 0$), lo cual es inviable en la realidad operativa y rompe la lógica de estiba aeroportuaria.

2.3.3 Resolución mediante “Gravedad Artificial” (Penalización de Coordenadas)

Para mitigar este comportamiento sin añadir restricciones complejas que ralenticen el modelo, se introdujo una "gravedad artificial" heurística en la función objetivo. Al restar la suma de las

coordenadas de anclaje de la caja ($x_i + y_i + z_i$), se fuerza una penalización matemática minúscula por cada unidad de distancia que la caja se aleje del origen. Esto rompe el empate entre los múltiples óptimos espaciales: el solver se ve matemáticamente presionado a empujar las cajas hacia el suelo ($z = 0$), hacia la pared frontal ($y = 0$) y hacia la izquierda ($x = 0$) para reducir la penalización al mínimo absoluto, logrando así una compactación automática de la carga.

2.3.4 Sintonización del Hiperparámetro

Es imperativo que la gravedad artificial no intervenga con la toma de decisiones económicas. Por ello, el hiperparámetro de penalización se fijó en $\lambda = 0.001$. Al ser un orden de magnitud drásticamente inferior al valor económico de cualquier mercancía en el inventario, se garantiza matemáticamente que el sistema jamás preferirá dejar una caja en tierra (sacrificar val_i) simplemente para ahorrar el costo marginal de sus coordenadas. La gravedad solo entra en vigor después de que el solver ha decidido qué cajas generan la máxima ganancia.

2.4. Restricciones Críticas

a) Unicidad y Capacidad: Cada caja solo puede ir en un compartimiento (o quedarse en tierra). El peso acumulado en el compartimiento j no debe superar $Wmax_j$:

$$\sum_{j \in J} V_{ij} \leq 1 \quad \forall i \in I$$

$$\sum_{i \in I} (w_i \cdot V_{ij}) \leq Wmax_j \quad \forall j \in J$$

```
def unicidad_rule(model, i): return sum(model.V[i, j] for j in model.J) <= 1
model.C_Unicidad = pyo.Constraint(model.I, rule=unicidad_rule)

def peso_max_rule(model, j): return sum(model.w[i] * model.V[i, j] for i in model.I) <= model.Wmax[j]
model.C_PesoMax = pyo.Constraint(model.J, rule=peso_max_rule)
```

b) Contención Geométrica y método "Big-M":

Las dimensiones sumadas a las coordenadas locales no pueden superar el tamaño del compartimiento. Solo se activan si la variable $V_{ij} = 1$:

$$x_i + dx_i \leq L_Xj + M(1 - V_{ij}) \quad \forall i, j$$

$$y_i + dy_i \leq L_Yj + M(1 - V_{ij}) \quad \forall i, j$$

$$z_i + dz_i \leq L_Zj + M(1 - V_{ij}) \quad \forall i, j$$

```
def b_x(m, i, j): return m.x[i] + m.dx[i] <= m.L_X[j] + m.M * (1 - m.V[i, j])
def b_y(m, i, j): return m.y[i] + m.dy[i] <= m.L_Y[j] + m.M * (1 - m.V[i, j])
def b_z(m, i, j): return m.z[i] + m.dz[i] <= m.L_Z[j] + m.M * (1 - m.V[i, j])
model.C_bx = pyo.Constraint(model.I, model.J, rule=b_x)
model.C_by = pyo.Constraint(model.I, model.J, rule=b_y)
model.C_bz = pyo.Constraint(model.I, model.J, rule=b_z)
```

c) No Traslapo (Restricción Disyuntiva): Para cualquier par de cajas i, k alojadas en el mismo compartimiento j , debe cumplirse al menos una relación espacial exclusiva. Se evita que los volúmenes se intersecten:

$$\begin{aligned} left_{ik} + right_{ik} + front_{ik} + back_{ik} \\ + below_{ik} + above_{ik} \\ \geq V_{ij} + V_{kj} - 1 \end{aligned}$$

```
def no_traslape(m, i, k, j): return (m.left[i, k] + m.right[i, k] + m.front[i, k] + m.back[i, k] + m.below[i, k] + m.above[i, k]) >= (m.V[i, j] + m.V[k, j] - 1)
model.C_NoTraslape = pyo.Constraint(model.Pares, model.J, rule=no_traslape)
```

d) Balance Estático (Centro de Gravedad):

El momento estático longitudinal (Suma del peso por su brazo de palanca absoluto) se linealizó para evitar no-linealidades (división entre variables). El balance total debe ser superior o

igual al límite mínimo multiplicando de forma cruzada el peso total:

$$\sum_{i \in I} (w_i \cdot Y_{abs_i}) \geq CG_{min} \cdot \sum_{i \in I} \sum_{j \in J} (w_i \cdot V_{ij})$$

(Se aplica una restricción simétrica análoga para CG_{min}).

```
def y_abs_min(m, i, j): return m.Y_abs[i] >= (m.Ycomp[j] + m.y[i] + (m.dy[i] / 2.0)) - m.M * (1 - m.V[i, j])
model.C_Yabs = pyo.Constraint(model.I, model.J, rule=y_abs_min)

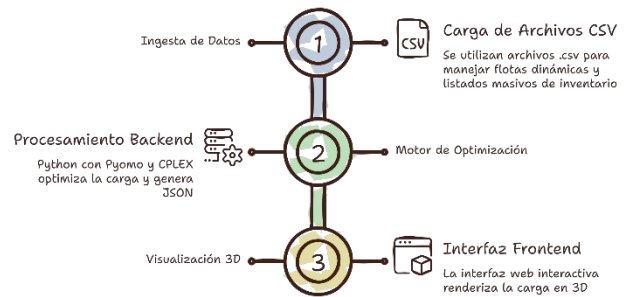
def y_abs_zero(m, i):
    return m.Y_abs[i] <= m.M * sum(m.V[i, j] for j in m.J)
model.C_Yabs_Zero = pyo.Constraint(model.I, rule=y_abs_zero)

def cg_min_rule(m): return sum(m.w[i] * m.Y_abs[i] for i in m.I) >= m.CG_min * sum(m.w[i] * m.V[i, j] for i in m.I for j in m.J)
model.C.CG_Min = pyo.Constraint(rule=cg_min_rule)

def cg_max_rule(m): return sum(m.w[i] * m.Y_abs[i] for i in m.I) <= m.CG_max * sum(m.w[i] * m.V[i, j] for i in m.I for j in m.J)
model.C.CG_Max = pyo.Constraint(rule=cg_max_rule)
```

III. METODOLOGÍA DE DATOS Y ARQUITECTURA DEL SOFTWARE

El sistema ha sido desarrollado bajo un paradigma de arquitectura desacoplada, separando estrictamente la capa de ingesta y optimización matemática (Backend) de la capa de renderizado interactivo (Frontend). Esta modularidad garantiza que el motor matemático pueda escalar de manera autónoma o cambiar de solver sin alterar la experiencia del usuario.



3.1. Pipeline de Ingesta y Estructuración de Datos (Pandas)

La flexibilidad del sistema radica en su capacidad para procesar flotas de aeronaves dinámicas y listados masivos de inventario a través de archivos planos. El backend en Python utiliza la librería Pandas para ejecutar un pipeline de preparación de datos estructurado en tres fases:

1. Filtrado y Validación de Flota:

Mediante `pd.read_csv`, el script lee las bases de datos de inventario y aeronaves. Se aplica un filtro condicional indexado para aislar los parámetros específicos del avión seleccionado (por ejemplo, AirbusA330), extrayendo sus límites globales de Centro de Gravedad (CG_{min} y CG_{max}). Si el identificador no coincide, el sistema arroja una excepción de valor estructurada para evitar ejecuciones en vacío.

2. Mapeo de Índices:

Se extraen las listas de claves primarias para las cajas disponibles (`id_caja`) y los compartimientos del avión (`id_comp`), las cuales actúan como los conjuntos discretos I y J dentro del modelo algebraico.

3. Conversión a Diccionarios Nativos:

Pyomo requiere estructuras de datos eficientes para la construcción de sus componentes de almacenamiento. Utilizando el método `.set_index().to_dict()` de Pandas,

los dataframes se transforman en diccionarios indexados por clave única. Este paso es crítico para vectorizar los parámetros físicos de las cajas ($dx_i, dy_i, dz_i, w_i, val_i$) y las dimensiones geométricas locales de los compartimientos ($L_{X_j}, L_{Y_j}, L_{Z_j}, W_{max_j}, Y_{comp_j}$).

3.2. Instanciación Dinámica del Modelo en Pyomo

Una vez estructurados los diccionarios, se inicializa un modelo determinista continuo (`pyo.ConcreteModel`). Los conjuntos y parámetros se inyectan dinámicamente utilizando los constructores `pyo.Set` y `pyo.Param`.

Un aspecto metodológico clave en el código es la creación del conjunto matricial de combinaciones `model.Pares`. Este componente genera pares ordenados estrictos (i, k) tales que $i < k$, mitigando la redundancia simétrica. Gracias a esta definición, las restricciones disyuntivas de no traslape espacial se formulan únicamente sobre combinaciones únicas de cajas, reduciendo a la mitad el número de variables binarias auxiliares de dirección (`left`, `right`, `front`, `back`, `below`, `above`) y aliviando significativamente la carga computacional del solver.

3.3. Configuración Avanzada y Orquestación del Solver

Para resolver la matriz MILP resultante, se vinculó el solver comercial CPLEX a través de la

interfaz de comunicación directa (`cplex_direct`). A diferencia de los solucionadores genéricos basados en archivos de texto intermedios (como los formatos LP o MPS), el modo directo optimiza la transferencia de la estructura del modelo en memoria, disminuyendo el tiempo muerto de preprocesamiento.

En entornos industriales o de operaciones aeroportuarias reales, el tiempo es un recurso crítico. Por este motivo, la llamada al solver se configuró con un diccionario de opciones explícito que define un límite de tiempo computacional estricto de 120 segundos (`options={'timelimit': 120}`). Si el solver no alcanza la convergencia absoluta en este intervalo debido a la complejidad NP-hard del inventario, CPLEX se detiene de forma segura y devuelve la mejor solución factible encontrada hasta el momento (con su respectiva brecha de optimalidad o GAP), garantizando que el sistema operativo de la aerolínea nunca sufra un bloqueo por bucle infinito.

3.4. Pipeline de Comunicación Backend-Frontend (Exportación JSON)

Una vez concluida la optimización, el backend no se limita a imprimir resultados en consola. Se ejecuta un algoritmo de recolección y formateo que interroga el estado final de las variables de decisión a través de la función `pyo.value()`:

- **Clasificación del Inventario:** El sistema evalúa la variable binaria de asignación V_{ij} . Si $V_{ij} > 0.5$, la caja se añade al arreglo de `cajas_cargadas`, adjuntando sus coordenadas espaciales calculadas (x, y, z) junto con sus dimensiones físicas. Si el bucle determina que la caja no fue asignada a ningún compartimiento, se clasifica automáticamente en el arreglo de `cajas_no_cargadas`.
- **Estructuración del Payload:** Toda esta información, junto con la geometría de los contenedores del avión y el valor económico total optimizado, se empaqueta en un diccionario unificado de Python.
- **Persistencia y Desacoplamiento:** El payload se serializa y exporta físicamente como un archivo JSON en la ruta estricta `interface/resultado_optimizacion.json`. Este archivo actúa como el contrato de comunicación o API de datos de la aplicación. El Frontend web interactivo lee este JSON de forma asíncrona para instanciar la escena y renderizar volumétricamente las cajas en sus posiciones exactas dentro del avión, u ordenarlas visualmente sobre la pista en la zona de carga rechazada.

IV. RESULTADOS Y PRUEBAS COMPUTACIONALES

Para evaluar el sistema, se realizaron pruebas de estrés utilizando perfiles de aeronaves como el AirbusA330 y un inventario mixto de alto volumen.

4.1. Análisis de Solución (Exploits Matemáticos mitigados)

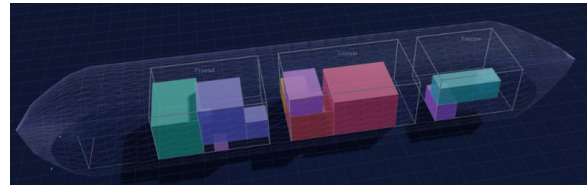
Durante las pruebas, se mitigaron comportamientos indeseados característicos de la heurística de los solvers:

- **Error de Tolerancia de Variables Continuas:** Se detectó que CPLEX aprovechaba la variable Big-M original ($M = 100,000$) permitiendo un traslape infinitesimal de las cajas. El valor se corrigió a $M = 50$, restaurando la impenetrabilidad de la geometría 3D.
- **Contrapeso Fantasma:** Se identificó que las cajas no seleccionadas aportaban al cálculo del momento de equilibrio. Se implementó una restricción estricta forzando $Y_{abs} = 0$ para toda carga en tierra.

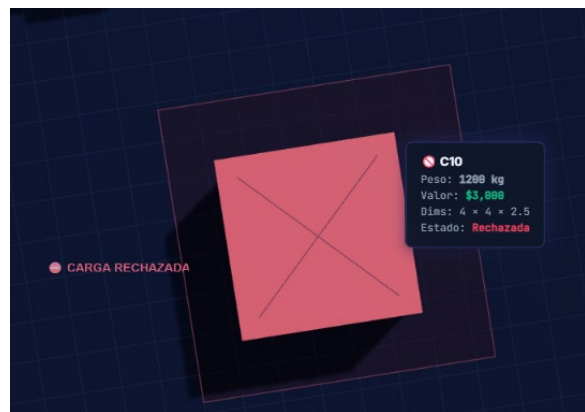
4.2. Caso de Estudio: Fallo Geométrico Exitoso (Caja C10)

En una prueba con 13 cajas disponibles por un valor total factible de más de \$13,000 USD, el sistema excluyó proactivamente la **Caja C10** (Peso: 1200, Valor: \$3000). Aunque el avión poseía holgura de peso ($>5,000$ u.) y la caja

poseía el mayor valor económico del inventario, la restricción geométrica de contención detectó que la caja poseía un ancho ($dx = 4.0$) superior al de las compuertas de la aeronave ($L_X = 3.5$).



El output de la solución determinó las coordenadas finales para las cajas admitidas, logrando compactar los volúmenes en el interior y obteniendo una ganancia neta optimizada de **\$10,249.98**, logrando un Centro de Gravedad perfecto de **15.00** (límite superior permitido del AirbusA330). Las mercancías incompatibles fueron renderizadas correctamente en la pista (área de carga rechazada).

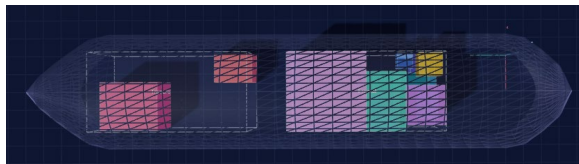


4.2. Caso de Estudio 2: Estrés de Balance Estático y Reposicionamiento (Centro de Gravedad)

Para verificar las inecuaciones de estabilidad longitudinal, se forzó un escenario utilizando el

Boeing 747, el cual posee límites estrictos de Centro de Gravedad entre $CG_{min} = 10.0$ y $CG_{max} = 18.0$. Al introducir cajas de alto peso volumétrico como la C6 (800 kg) y C8 (400 kg), una asignación heurística tradicional (por ejemplo, First-Fit) habría llenado el primer espacio disponible: el compartimiento Frontal ($Y_{comp} = 4.0$). Si el modelo hubiera hecho esto, el Y_{abs} de la carga generaría un momento estático inferior a 10.0, volviendo a la aeronave peligrosamente "pesada de nariz".

La solución del modelo MILP distribuyó estratégicamente las cargas hacia el compartimiento Trasero ($Y_{comp} = 14.0$), compensando el brazo de palanca y elevando el promedio del CG final a un rango seguro y operable para el despegue.



4.3. Caso de Estudio 3: Alta Densidad de Valor y Compactación Espacial (Knapsack)

El tercer escenario demostró el comportamiento cruzado del empaquetado y la mochila (3D Bin Packing + Knapsack). El inventario presentaba mercancías muy voluminosas, pero de bajo valor como la C4 ($dx = 3.0, dy = 2.0, dz = 2.0$, Valor: \$150), en contraste con mercancías diminutas de altísimo valor como la C3 ($0.5 \times 0.5 \times 0.5$, Valor: \$1,200) y la C11 ($0.6 \times 0.6 \times 0.5$, Valor: \$2,000). El solver optimizó la

densidad de carga (m^3) priorizando la ingesta de las cajas C3 y C11. Adicionalmente, gracias a la penalización por "Gravedad Artificial" implementada en la función objetivo, estas cajas pequeñas no quedaron dispersas por el compartimiento, sino que fueron ancladas algorítmicamente en las esquinas inferiores del espacio de carga (minimizando las variables x, y, z), abriendo volumen útil para el resto del inventario.

V. CONCLUSIONES

El modelo formulado es altamente capaz de resolver de manera autónoma las problemáticas logísticas del empaquetado 3D aeronáutico. La integración de la Programación Lineal Entera Mixta para la toma de decisiones complejas superó las limitantes de los algoritmos heurísticos tradicionales, demostrando un acatamiento estricto a las restricciones geométricas, espaciales y de seguridad en la aviación. La plataforma final es funcional, escalable para distintos tipos de aeronaves y provee una experiencia visual que clarifica la lógica matemática subyacente.

